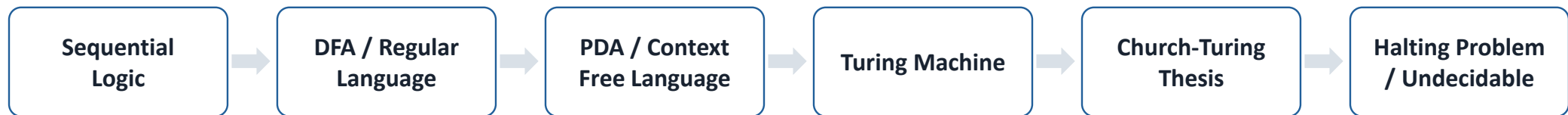
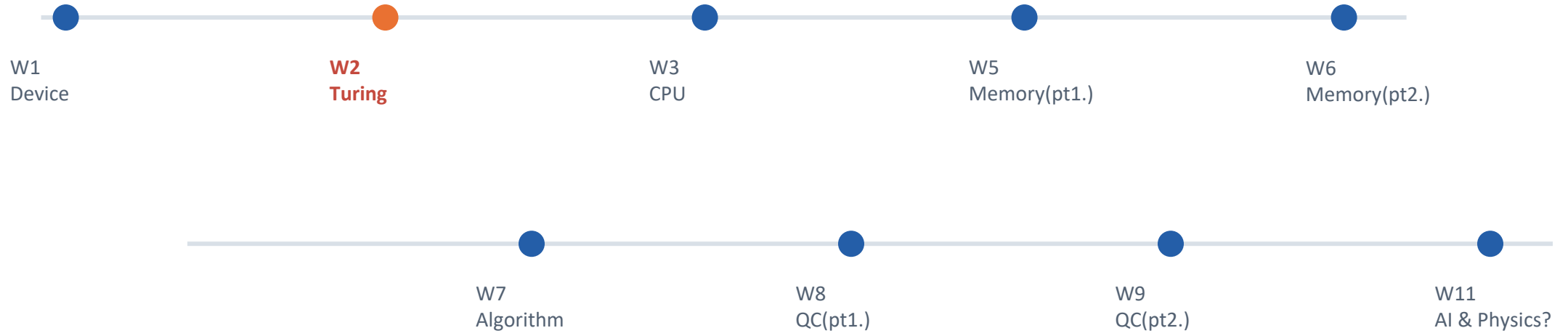


The Power and Limits of Computation

Jaehyun Choi

Department of Computer Science and Engineering
Seoul National University

RoadMap



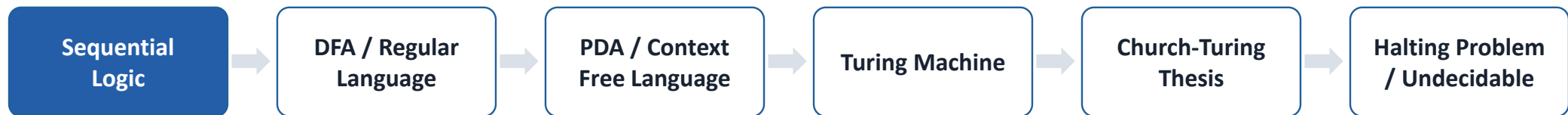
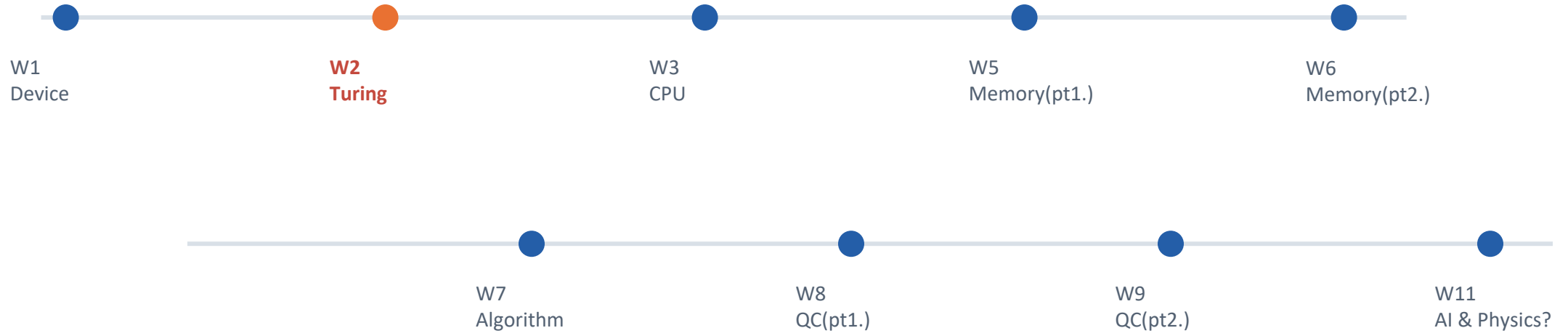
Key Questions

- What is computation?
- What can computers compute?
- Are there problems that no computer can solve?

Difficult vs. Impossible

- Can Every Well-Defined Problem Be Solved by a Computer?
 - What is the length of the shortest simple path in a given non-negative weighted graph?
 - What is the length of the longest simple path in a given non-negative weighted graph?
 - Does a given program terminate?
 - Are two programs equivalent?
- Some problems are computationally intractable
- Some decision problems are undecidable

RoadMap



Combinational Logic

- Combinational Circuits
 - The output depends only on the current input
 - There is no memory of previous inputs
 - They are built from Boolean logic gates
- Examples
 - Adders, Multiplexers, Comparators, Decoders
- Output = $F(\text{Inputs})$

Adding Memory

- Sequential Circuits
 - The output depends on the current input and stored state.
 - Flip-flops store bits.
 - A clock controls state updates.
- Next state = $F(\text{Current state}, \text{Inputs})$
- Output = $G(\text{Current state}, \text{Inputs})$

Finite State Machine : Traffic Light Controller

- States

- q_R : Red

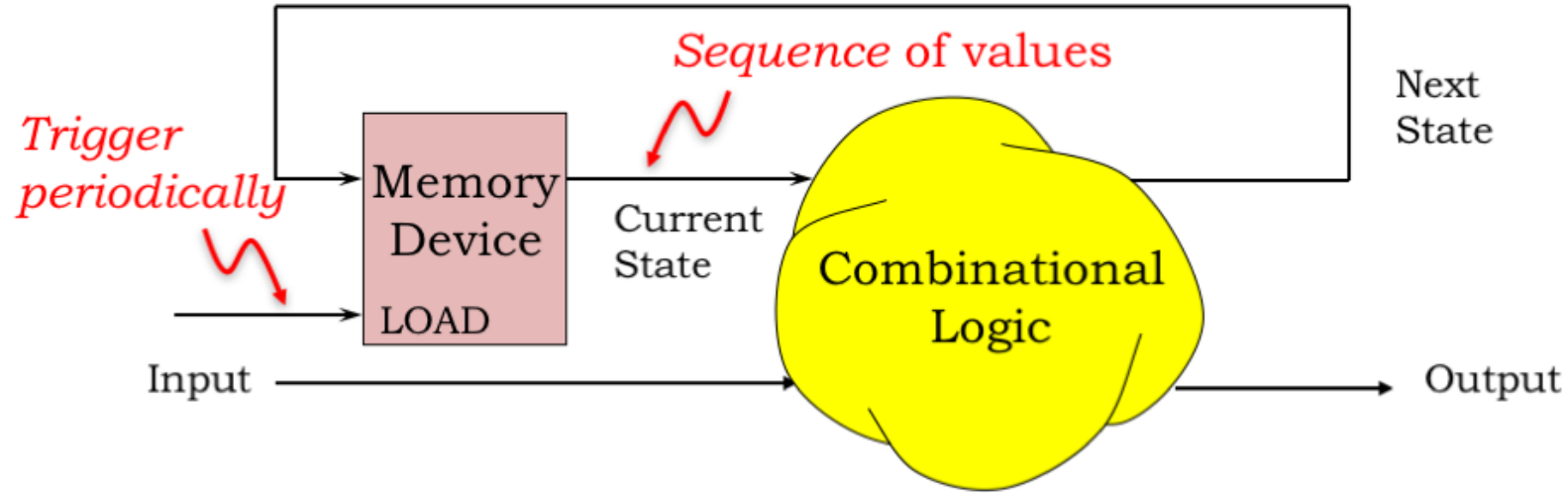
- q_Y : Yellow

- q_G : Green

- Transitions

- $q_R \rightarrow q_G \rightarrow q_Y \rightarrow q_R$

Hardware Structure of FSM



- State register: stores the current state
- Next-state logic: computes the next state
- Output logic: computes the output

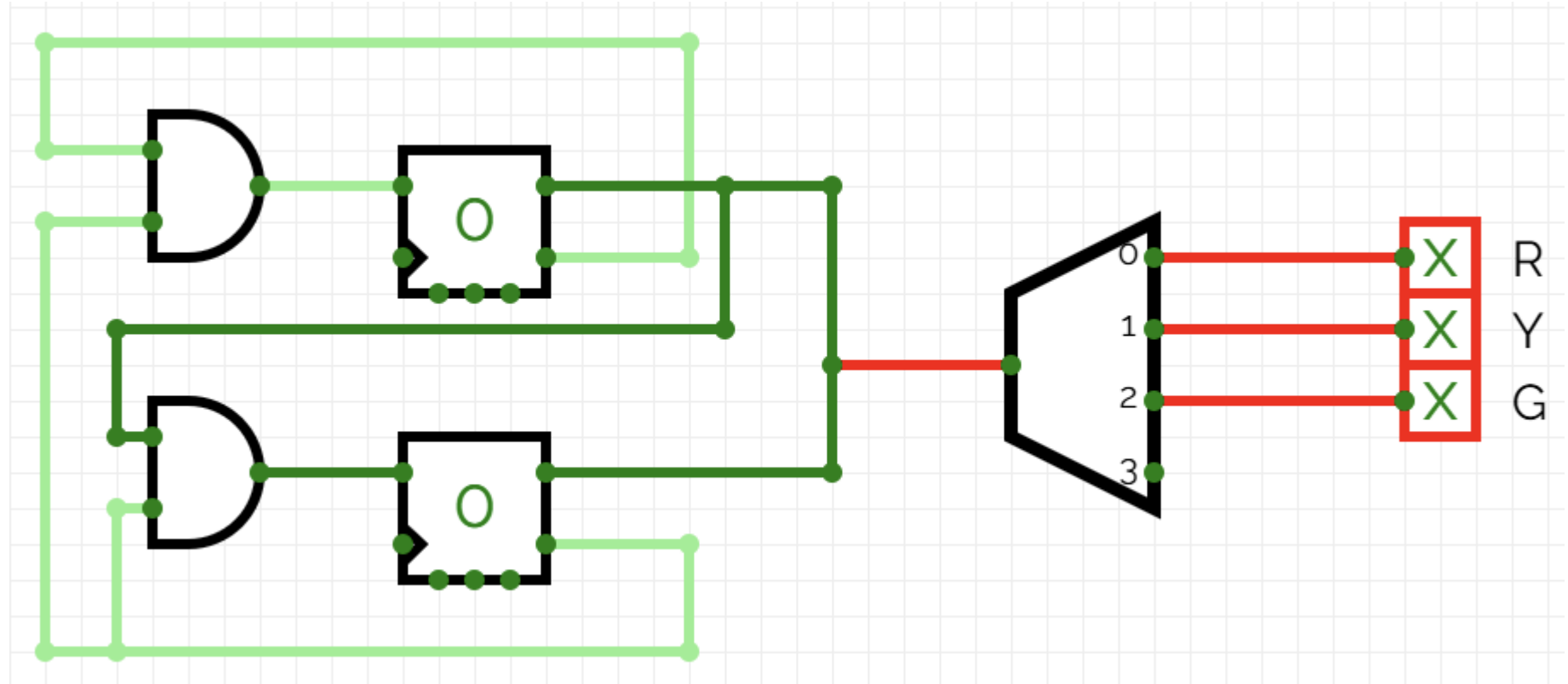
Hardware Structure : Traffic Light Controller

- State encoding

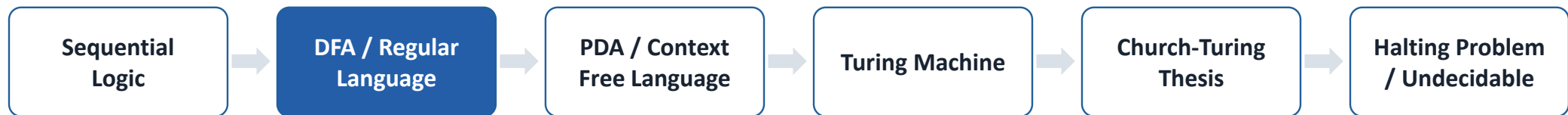
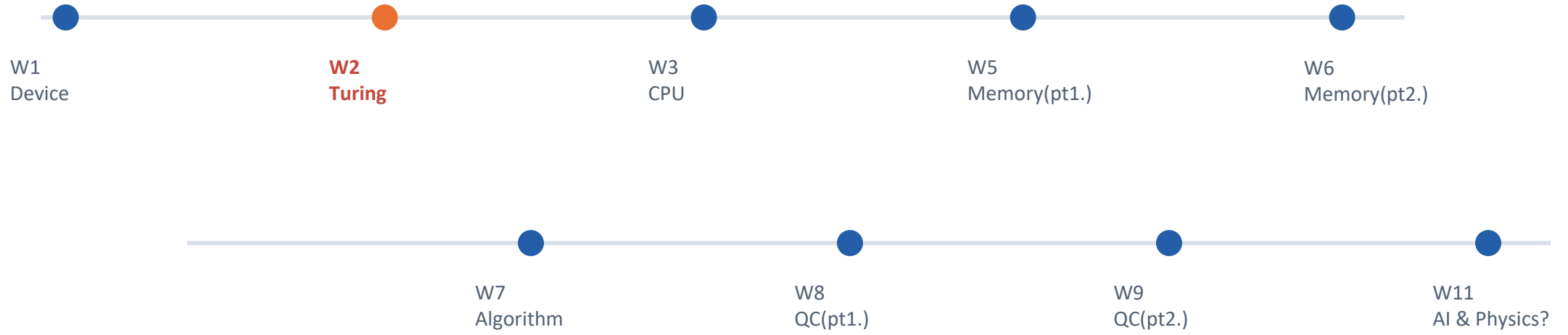
- $q_R : 00$

- $q_Y : 01$

- $q_G : 10$



RoadMap

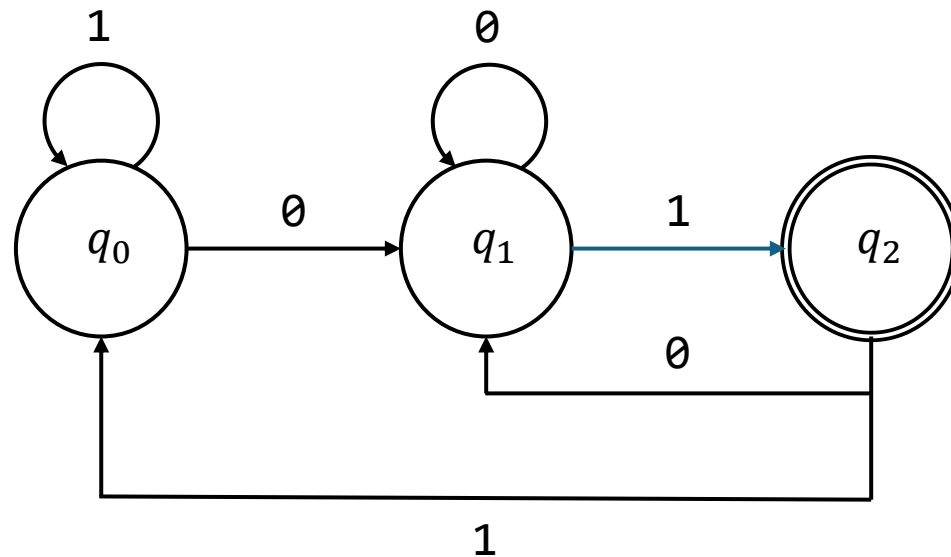


Formal Definition of a DFA

- Deterministic Finite Automaton
 - $M = (Q, \Sigma, \delta, q_0, F)$
 - Q : a finite set of states
 - Σ : the input alphabet
 - δ : the transition function
 - q_0 : the start state
 - F : the set of accepting states
- $\delta(\text{current state}, \text{input symbol}) = \text{next state}$
- A string is accepted if the machine finishes in a state in F

Example DFA

- Binary Strings Ending in 01
 - $L = \{w \in \{0, 1\}^* \mid w \text{ ends in } 01\}$
- State meaning
 - q_0 : no useful suffix
 - q_1 : the last symbol was 0
 - q_2 : the current suffix is 01



Regular Languages

- A language is regular if some finite automaton recognizes it
- Equivalent representations
 - DFA
 - NFA
 - Regular expressions
 - Regular grammars
- Different representations, equal expressive power

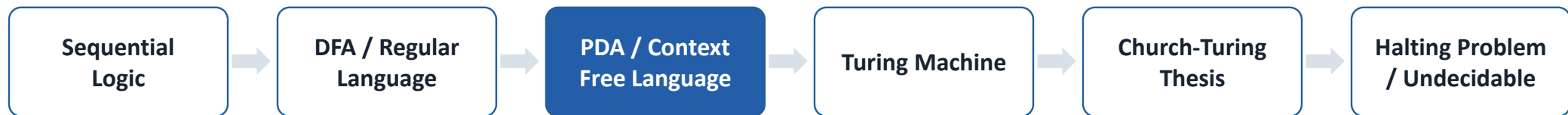
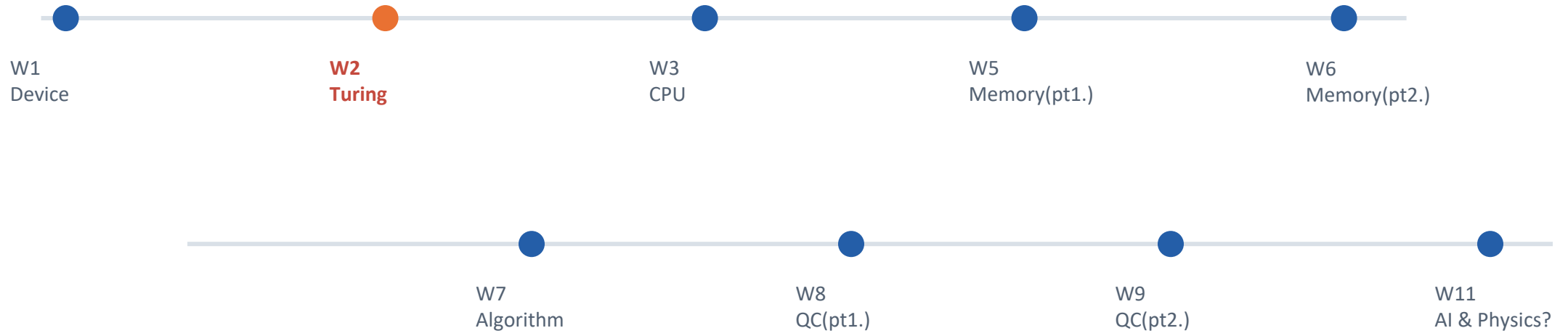
The Limitation of DFA

- Can a Finite Automaton Recognize This Language?
 - $L = \{0^n 1^n \mid n \geq 1\}$
- The machine must remember: How many 0's have been read?
- A finite automaton has only finitely many states
 - By Pigeonhole principle, DFA can't recognize $L = \{0^n 1^n \mid n \geq 1\}$

Quiz 1: DFA

- Determine whether there exists a DFA that recognizes the following language. If so, construct such a DFA.
- $L_1 = \{w \in \{0, 1\}^* : (N_0(w) \bmod 2) \geq (N_1(w) \bmod 2)\}$
- $L_2 = \{w \in \{0, 1\}^* : \text{the binary value of } w \text{ is divisible by } 5\}$
- $L_3 = \{w \in \{0, 1\}^* : N_0(w) = N_1(w)\}$

RoadMap



Generating Languages

- Context-Free Grammar
 - $G = (V, \Sigma, R, S)$
 - V : variables
 - Σ : alphabets
 - R : production rules
 - S : starting variable
- A grammar generates strings by repeatedly applying production rules

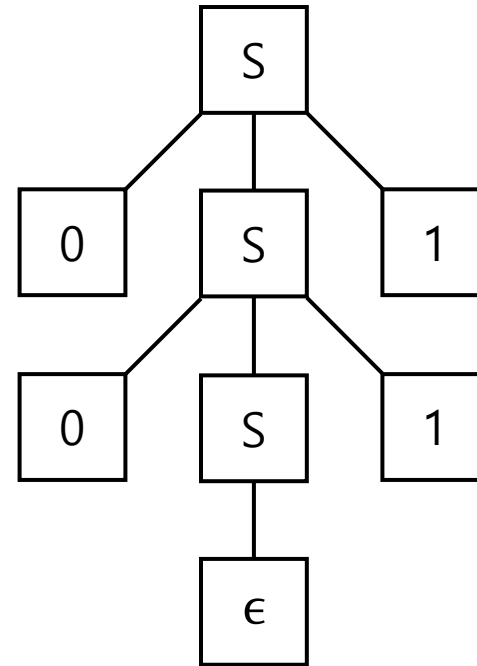
Parse Trees

- Parse Tree for 0011

- Grammar

- $S \rightarrow 0S1$

- $S \rightarrow \epsilon$



- A parse tree represents the hierarchical structure of a derivation

Ambiguous Grammar

- A CFG is ambiguous if some string has two distinct parse trees
- Example ambiguous grammar
 - $S \rightarrow S + S$
 - $S \rightarrow S * S$
 - $S \rightarrow id$
- String : $id + id * id$
 - $(id + id) * id$
 - $id + (id * id)$

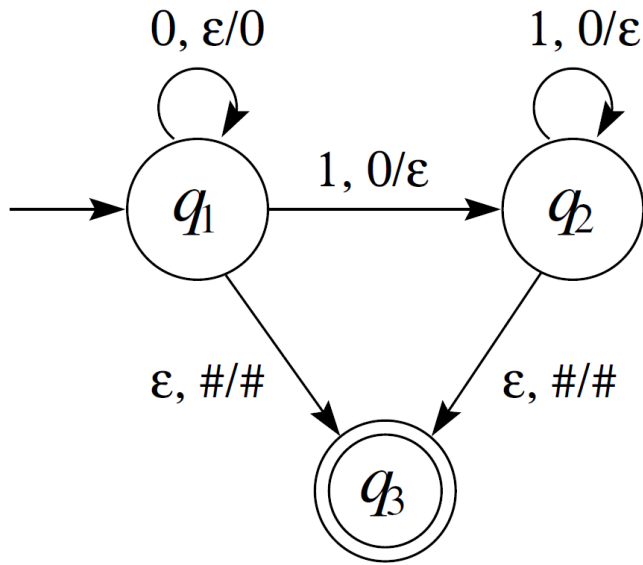
Formal Definition of PDA

- Pushdown Automaton

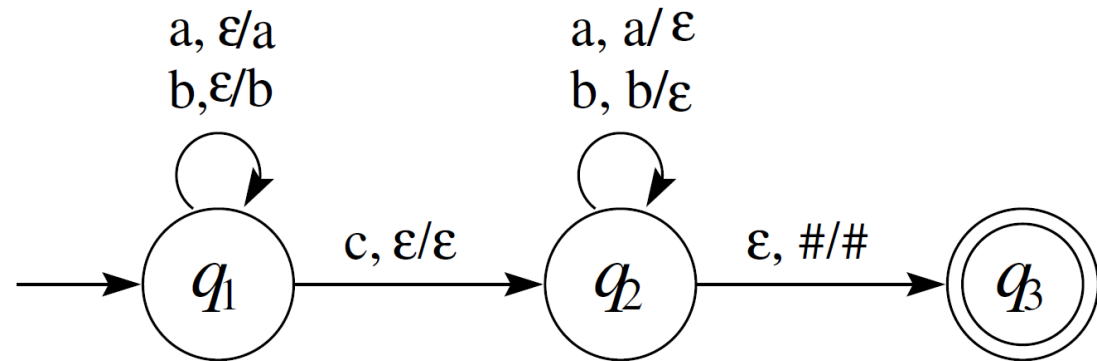
- $M = (Q, \Sigma, \Gamma, \Delta, q_0, F)$
- Q : a finite set of states
- Σ : the input alphabet
- Γ : the stack alphabet, including the initial stack symbol #
- Δ : the transition function
- q_0 : the start state
- F : the set of accepting states
- $\Delta(\text{current state}, \text{input symbol}, \text{stack symbol}) = (\text{next state}, \text{stack symbols})$

Example PDA

➤ $L = \{0^n 1^n \mid n \geq 0\}$



➤ $L = \{wcw^R \mid w \in \{a, b\}^*\}$



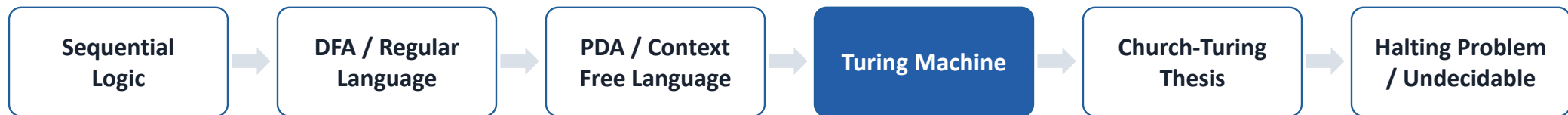
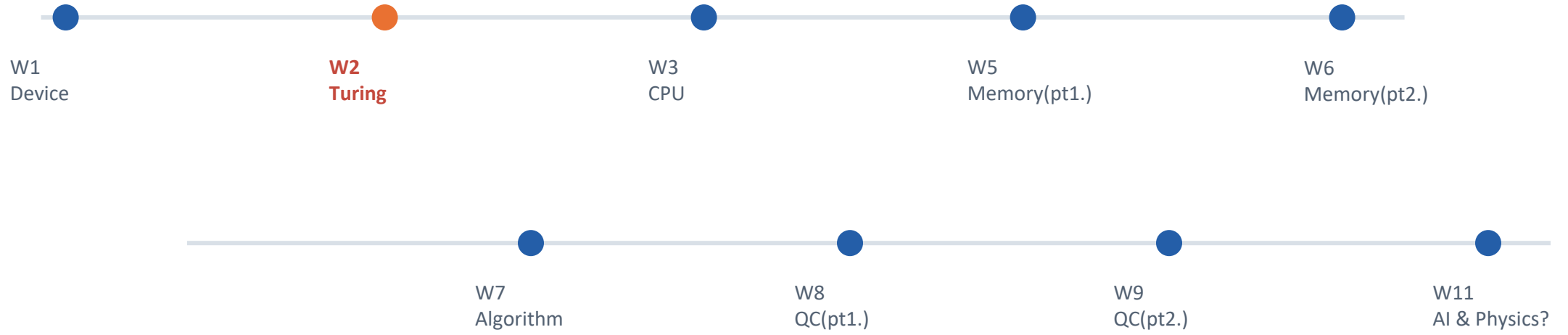
The Limitation of PDA

- Can a Pushdown Automaton Recognize This Language?
 - $L = \{0^n 1^n 2^n \mid n \geq 1\}$
- A single stack can compare
 - number of 0's with number of 1's
- But it cannot also preserve enough information to compare
 - number of 1's with number of 2's

Quiz 2: CFG and PDA

- Construct a CFG that generates all well-formed parenthesis strings, and determine whether the grammar is ambiguous.
- $L = \{0^n 1^m \mid n \leq m \leq 2n\}$
 - Construct a context-free grammar for L
 - Construct a PDA that recognizes L

RoadMap



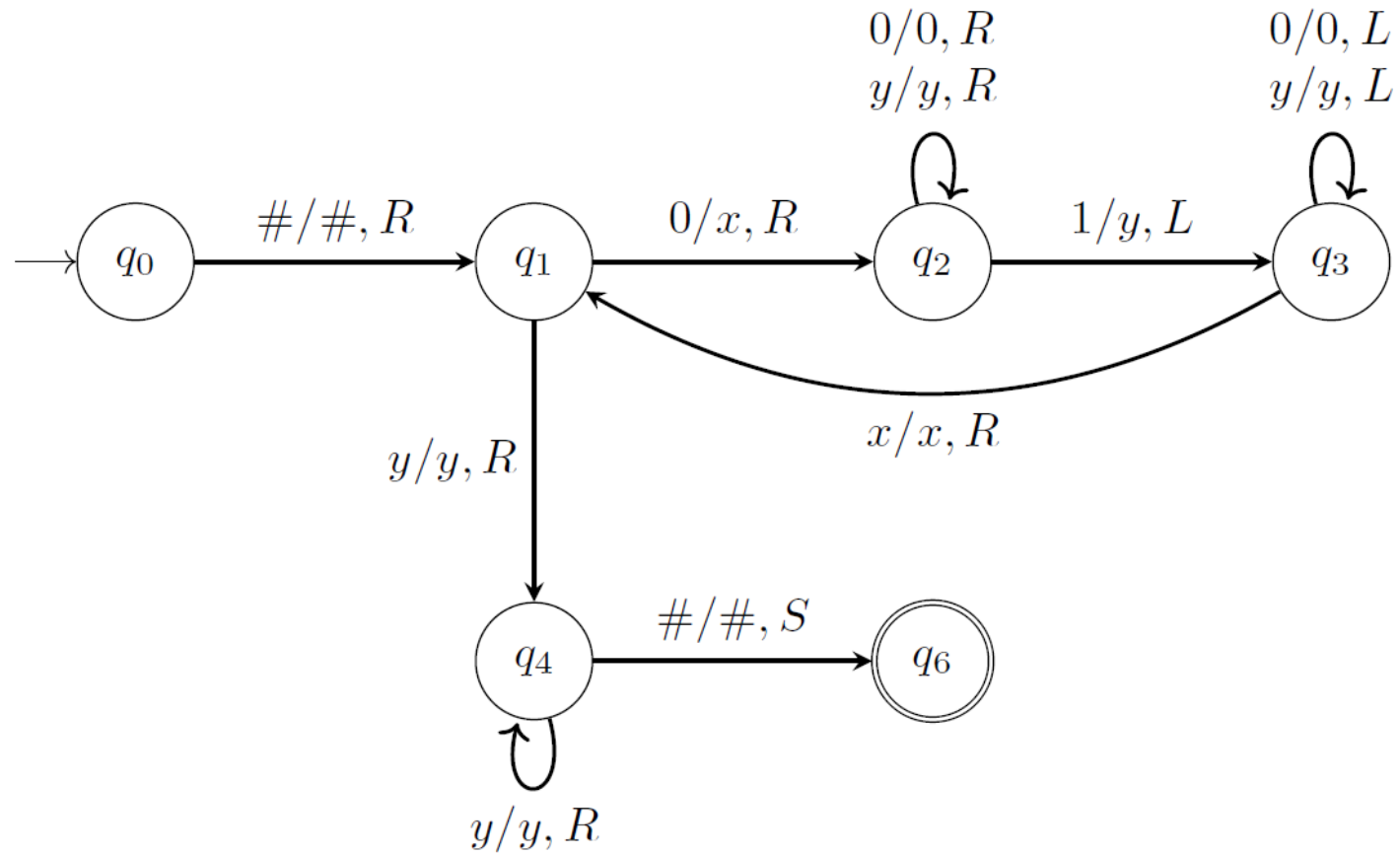
Formal Definition of TM

- Turing Machine

- $M = (Q, \Sigma, \Gamma, \delta, q_0, H)$
- Q : a finite set of states
- Σ : the input alphabet
- Γ : the tape alphabet
- δ : the transition function
- q_0 : the start state
- H : the set of halting states
- $\delta(\text{current state}, \text{scanned tape symbol}) = (\text{next state}, \text{tape symbol}, \text{direction})$

Example TM

➤ $L = \{0^n 1^n \mid n \geq 1\}$



Modern CPU



**Based on figures from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

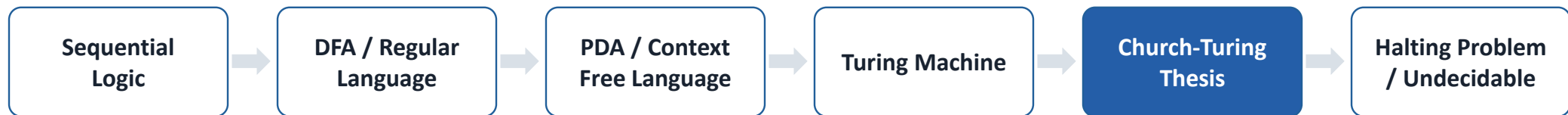
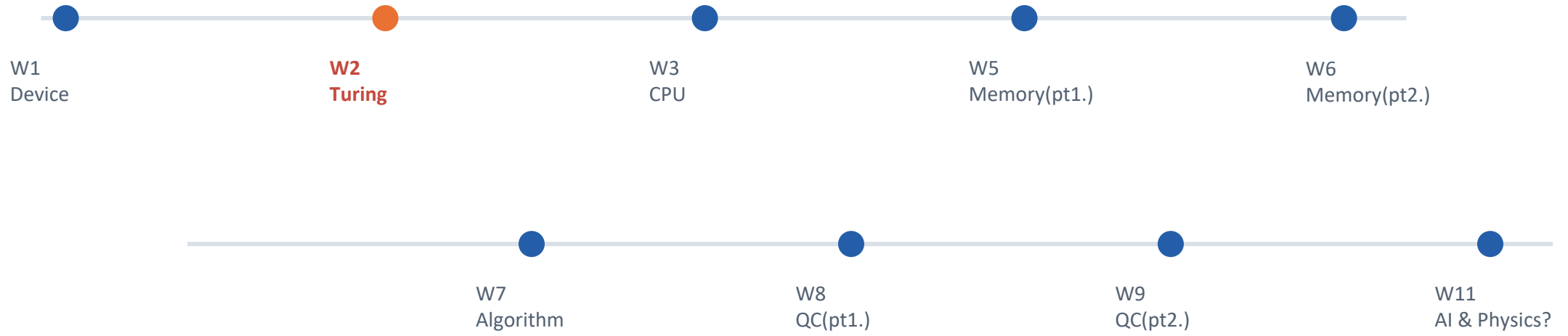
Turing Machine and Modern CPU (1)

Turing Machine	Modern CPU	Interpretation
Tape	Memory	Stores input, intermediate data, and output.
Tape Cell	Memory Location	Stores one symbol or data value.
Tape-Head Position	Current Memory Address	Identifies the location currently being accessed.
Read / Write	Memory Access	Corresponds to load and store operations.
Finite Control State	Instruction and Control State	Represents the current step of execution.
Transition Function	Execution and Control Logic	Determines the next operation and state.

Turing Machine and Modern CPU (2)

- Why Can a Turing Machine Simulate a CPU?
 - CPU configuration can be encoded on a Turing-machine tape
 - PC, Registers, Memory, Program
- A Turing machine can simulate a CPU
 - Locate the current instruction
 - Read the required operands
 - Compute the result
 - Update registers and memory
 - Update the program counter

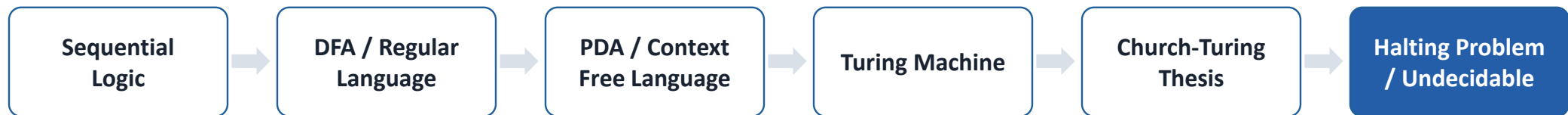
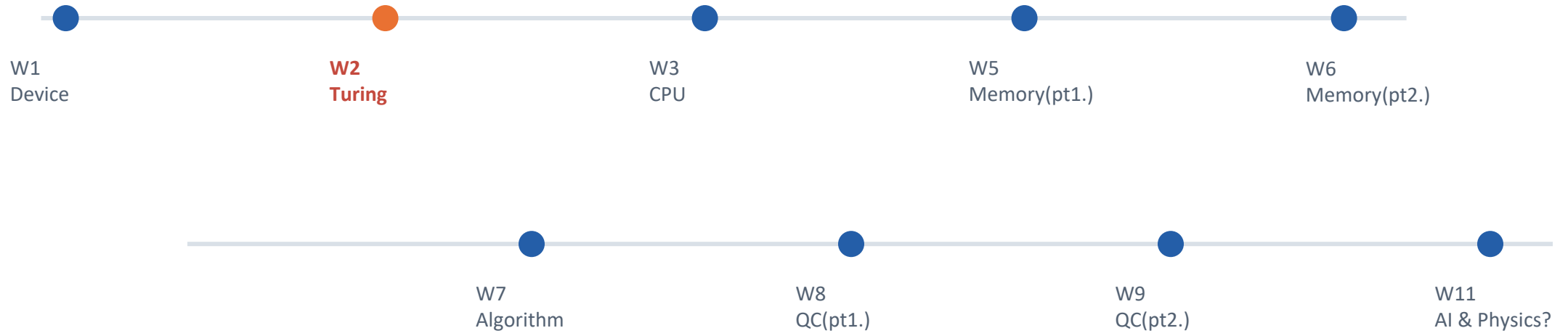
RoadMap



Church-Turing Thesis

- Any function that can be computed by an algorithm can be computed by a Turing machine
- Key Idea
 - Algorithmic Computation = Turing-Machine Computation
- The Church–Turing Thesis defines the boundary of what can be computed algorithmically

RoadMap



Encoding a Turing Machine

- Turing machine has a finite description
 - A finite set of states
 - A finite tape alphabet
 - A finite transition table
- TM can be encoded as a finite binary string
 - $\langle M \rangle$: The encoding of Turing machine M
 - $\langle M, w \rangle$: Turing machine M with its input w

Universal Turing Machine

- A universal Turing machine U receives $\langle M, w \rangle$
- Simulates machine M running on input w
- Programs can be treated as data

Halting Problem

- Input
 - $\langle M, w \rangle$
- Question
 - Will machine M eventually halt when run on input w ?
- Undecidable

Reduction of Language

- A reduction from a language L_1 to a language L_2
 - Computable function f s.t. $w \in L_1 \Leftrightarrow f(w) \in L_2$

Post Correspondence Problem

- Input

- $P = ((u_1, v_1), \dots, (u_n, v_n))$

- P correspondence

- $\exists i_1, \dots, i_r \ (r \geq 1) \text{ s.t. } u_{i_1} \cdots u_{i_r} = v_{i_1} \cdots v_{i_r}, \text{ indices may be repeated}$

- Question

- P correspondence exists?

- Undecidable

- Example $P = ((a, ab), (ba, a))$

Quiz 3: Why Undecidable?

- Given two context-free grammars G_1 and G_2 , is $L(G_1) \cap L(G_2) = \phi$?
- Given a context-free grammar G , does $L(G)$ contain a palindrome?
- Given $\langle M \rangle$, does M halt on ϵ ? $L_\epsilon = \{\langle M \rangle \mid \epsilon \in L(M)\}$
- Given $\langle M \rangle$, is there no input on which M halts? $L_\phi = \{\langle M \rangle \mid L(M) = \phi\}$
- Given $\langle M_1 \rangle$, $\langle M_2 \rangle$, deciding whether $L(M_1) = L(M_2)$

Rice's theorem

- Let C be a non empty proper subset of recursively enumerable languages
- $L_C = \{\langle M \rangle \mid L(M) \in C\}$ is undecidable

Undecidable Problems and Quantum Computers

- Qubits

- $\alpha|0\rangle + \beta|1\rangle$

- Quantum program consists of a finite sequence of quantum gates
- In the standard model, each gate has a computable specification
- Turing machine can simulate these gate operations to any required finite precision
- Quantum computers may change how fast a problem can be solved, but they do not make undecidable problems decidable

References

- [1] Kunsoo Park, **Lecture Materials for Automata Theory**, Seoul National University
- [2] D.R. Jeong, **Lecture Materials for Computer Architecture**, Seoul National University
- [3] Michael A. Nielsen and Isaac L. Chuang, **Quantum Computation and Quantum Information**,
Cambridge University Press